

Binary Search & Its Variations

A complete guide to binary search and related algorithms for placements, online assessments, and competitive programming.

1. Basic Binary Search

1. Overview

Binary search is an algorithm that finds the position of a target value in a **sorted** array. It works by repeatedly dividing the search interval in half. If the target is less than the middle element, search the left half; otherwise, search the right half. This halves the search space at every step.

2. Intuition

Simple explanation: Imagine looking up a word in a physical dictionary. You don't start at page 1 and flip page by page. Instead, you open the dictionary roughly in the middle. If the word you want comes alphabetically after the page you opened, you ignore the entire left half and search only the right half. You repeat this — each time cutting the remaining pages in half — until you find the word.

Why it works: Because the array is sorted, every comparison eliminates half the remaining elements. This gives logarithmic time — extremely fast even for huge arrays.

Step-by-step reasoning:

1. Start with the entire sorted array.
2. Look at the middle element.
3. If it matches the target — done.
4. If the target is smaller, the answer must be in the left half. Discard the right half.
5. If the target is larger, the answer must be in the right half. Discard the left half.
6. Repeat until the target is found or the search space is empty.

3. When to Use It

- The input is **sorted** (ascending or descending).
- You need to find an element or a position in a monotonic sequence.
- The search space is large and you need $O(\log n)$ time.
- You need to find a boundary point where a condition changes from false to true (or true to false).

Common trigger phrases:

- "Given a sorted array ..."
- "Find an element in ..."
- "Search in ..."
- "Sorted list/array/sequence"
- " $O(\log n)$ time"

4. When Not to Use It

- The array is **not sorted**. Sorting first costs $O(n \log n)$, which may be too expensive.
- You only need to search a few times on a small array. Linear search is simpler.
- The data structure doesn't support random access (e.g., linked list). Binary search requires $O(1)$ index access.
- All elements are the same as the target — still works, but any element works.
- The array is very small ($n < 10$) — linear search is fine and simpler.

5. Core Concepts

Concept	Explanation	Why It Matters
Monotonic predicate	A condition that is false for some prefix and true for the rest (or vice versa).	Binary search works on any monotonic function, not just sorted arrays.
Search space	The range of indices or values being searched.	Must be clearly defined and finite.
Midpoint	The middle index of the current search space.	Choosing mid correctly avoids infinite loops.
Invariant	A condition that stays true throughout the search.	Helps prove correctness. E.g., "answer always lies between lo and hi."
Narrowing	The process of moving lo or hi to mid.	Must ensure the search space strictly shrinks.

6. Step-by-Step Algorithm

Standard binary search (find exact target in sorted array):

1. Initialize `lo = 0`, `hi = n - 1`.
2. While `lo <= hi`:
 - `mid = lo + (hi - lo) / 2` (avoid overflow).
 - If `arr[mid] == target`, return `mid`.
 - If `arr[mid] < target`, set `lo = mid + 1` (search right half).
 - Else set `hi = mid - 1` (search left half).
3. If loop ends, return `-1` (not found).

7. Dry Run

Input: `arr = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72]`, `target = 23`

Step	lo	hi	mid	arr[mid]	Comparison	Action
1	0	9	4	16	$16 < 23$	<code>lo = 5</code>
2	5	9	7	45	$45 > 23$	<code>hi = 6</code>
3	5	6	5	23	$23 == 23$	Found at index 5

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Returns index of target if found, else -1
int binarySearch(const vector<int>& arr, int target) {
    int lo = 0, hi = (int)arr.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // avoids overflow
        if (arr[mid] == target)
            return mid;
        if (arr[mid] < target)
            lo = mid + 1;
        else
            hi = mid - 1;
    }
    return -1;
}

// Example usage
int main() {
    vector<int> arr = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72};
    int idx = binarySearch(arr, 23);
    cout << idx << endl; // Output: 5
    return 0;
}
```

9. Python Implementation

```
def binary_search(arr, target):
    """Returns index of target if found, else -1."""
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2 # avoids overflow
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

# Example usage
arr = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72]
print(binary_search(arr, 23)) # Output: 5
```

10. Code Explanation

- **lo = 0, hi = n-1**: The search space is the entire array.
- **while (lo <= hi)**: Continue while the search space is non-empty. **<=** ensures single-element arrays are handled.
- **mid = lo + (hi - lo) / 2**: Calculate midpoint. Using **lo + (hi - lo)/2** instead of **(lo + hi)/2** prevents integer overflow when **lo** and **hi** are large.
- **if (arr[mid] == target) return mid**: Target found.
- **else if (arr[mid] < target) lo = mid + 1**: Target is in the right half. We can safely exclude **mid** because we already checked it.
- **else hi = mid - 1**: Target is in the left half.
- **return -1**: Target not found after exhausting the search space.

Key insight: Each iteration discards at least half the remaining elements, giving $O(\log n)$ time.

11. Complexity Analysis

Operation	Complexity
Best case (target at mid)	$O(1)$
Average/Worst case	$O(\log n)$
Space complexity (iterative)	$O(1)$
Space complexity (recursive)	$O(\log n)$ due to call stack

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Exact match	Find exact element in sorted array	Standard binary search	LeetCode 704
Find boundary	Find first/last position where condition changes	Modified binary search keeping invariant	LeetCode 34
Search in range	Answer lies in some monotonic range	Binary search on answer space	LeetCode 875
Unknown size	No bound given on array size	Exponential backoff + binary search	LeetCode 702

13. Common Mistakes

- **Off-by-one in loop condition**: Using **lo < hi** instead of **lo <= hi** can miss elements.
- **Not updating mid correctly**: **mid = (lo + hi) / 2** can overflow for large ints.
- **Infinite loops**: When **lo = mid** instead of **lo = mid + 1** with **lo < hi** condition.
- **Forgetting to sort**: Binary search only works on sorted data.
- **Wrong search space**: Not including all possible values.
- **Integer division rounding**: Mid calculation in negative ranges needs attention.

14. Edge Cases

- **Empty array:** Should return -1 immediately.
- **Single element:** Must work correctly (loop runs once).
- **Target not present:** Returns -1.
- **Target at first or last position:** Handled correctly.
- **Duplicate values:** Standard binary search returns any one occurrence (implementation-dependent).
- **All elements same:** Still works; returns some occurrence.
- **Very large arrays:** Use `mid = lo + (hi - lo) / 2` to avoid overflow.

15. Variations

Variation	What Changes	When to Use	Importance
Lower bound	Find first position $\geq$ target	Sorted arrays, insertion points	High
Upper bound	Find first position $>$ target	Range queries, counting	High
First occurrence	Find leftmost occurrence	When duplicates matter	High
Last occurrence	Find rightmost occurrence	When duplicates matter	High
Ternary search	Divide into 3 parts instead of 2	Unimodal functions	Medium

16. Related Algorithms

Algorithm	Connection	How to Choose
Linear search	Simpler, no sorting requirement	Use for small arrays or unsorted data
Exponential search	First finds range, then binary searches	Use when array size is unknown/infinite
Interpolation search	Uses value distribution to guess position	Use for uniformly distributed sorted data
Ternary search	Divides into 3 parts for unimodal functions	Use for finding max/min of unimodal functions

2. Lower Bound

1. Overview

Lower bound finds the **first position** in a sorted array where the value is $\geq$ **target**. If all elements are less than target, it returns `n` (the index just past the end). This is equivalent to C++'s `std::lower_bound`.

2. Intuition

Think of finding the first seat in a row where a person of at least a certain height can sit. You scan from left to right conceptually, but binary search lets you skip most of the row. The first person who meets the height requirement is your answer. If nobody meets it, the answer is the "end" of the row.

Why it works: The array is sorted. Once we find a position where the value is $\geq$ target, all positions to the right also satisfy the condition. So we keep narrowing down to find the leftmost such position.

3. When to Use It

- You need to find the insertion point for a value in a sorted array.
- You need to find the first element not less than a given value.
- You need to implement "ceiling" of a number.
- Range queries: counting elements in [L, R] range.

Common trigger phrases:

- "First element greater than or equal to ..."
- "Lower bound of ..."
- "Insert position"
- "Ceiling of ..."
- "Count elements less than ..."

4. When Not to Use It

- Unsorted data — doesn't make sense.
- You need exact match only — use standard binary search.
- You need the first element strictly greater — use upper bound instead.

5. Core Concepts

Concept	Explanation
Invariant	$arr[lo] < target$ for the left portion, $arr[hi] \geq target$ for the right portion
Answer range	Answer lies in $[0, n]$ (n means no element satisfies)
Mid calculation	Same as binary search, but narrowing logic differs

6. Step-by-Step Algorithm

1. Initialize $lo = 0$, $hi = n$. (Note: hi is n , not $n-1$, because answer can be n .)
2. While $lo < hi$:
 - $mid = lo + (hi - lo) / 2$.
 - If $arr[mid] < target$, set $lo = mid + 1$ (answer is to the right).
 - Else set $hi = mid$ (answer could be mid or to the left).
3. Return lo .

7. Dry Run

Input: $arr = [1, 3, 5, 7, 9, 11]$, $target = 6$

Step	lo	hi	mid	arr[mid]	Comparison	Action
1	0	6	3	7	$7 \geq 6$	$hi = 3$
2	0	3	1	3	$3 < 6$	$lo = 2$
3	2	3	2	5	$5 < 6$	$lo = 3$

Step	lo	hi	mid	arr[mid]	Comparison	Action
4	3	3	—	—	lo == hi	Stop

Result: `lo = 3`, and `arr[3] = 7` is the first element ≥ 6 .

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Returns first index where arr[idx] >= target
// Returns n if no such element exists
int lowerBound(const vector<int>& arr, int target) {
    int lo = 0, hi = (int)arr.size(); // hi = n, not n-1
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] < target)
            lo = mid + 1;
        else
            hi = mid;
    }
    return lo;
}

// Example usage
int main() {
    vector<int> arr = {1, 3, 5, 7, 9, 11};
    cout << lowerBound(arr, 6) << endl; // Output: 3 (arr[3] = 7)
    cout << lowerBound(arr, 1) << endl; // Output: 0 (arr[0] = 1)
    cout << lowerBound(arr, 12) << endl; // Output: 6 (n, no element)
    return 0;
}
```

9. Python Implementation

```
def lower_bound(arr, target):
    """Returns first index where arr[idx] >= target.
    Returns len(arr) if no such element exists."""
    lo, hi = 0, len(arr)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo

# Example usage
arr = [1, 3, 5, 7, 9, 11]
```

```
print(lower_bound(arr, 6))    # Output: 3
print(lower_bound(arr, 1))    # Output: 0
print(lower_bound(arr, 12))   # Output: 6
```

10. Code Explanation

- **hi = n**: Unlike standard binary search where **hi = n-1**, lower bound uses **hi = n** because the answer could be **n** (no element $\geq$ target).
- **while (lo < hi)**: We stop when **lo == hi**. At that point, both point to the first position where **arr[idx] $\geq$ target**.
- **if (arr[mid] < target) lo = mid + 1**: **arr[mid]** is too small. The answer must be to the right, and we can safely exclude **mid**.
- **else hi = mid**: **arr[mid]** is $\geq$ target. So **mid** is a candidate, but there could be an earlier occurrence. We keep **mid** in the search space.

Key difference from standard binary search: When **arr[mid] $\geq$ target**, we set **hi = mid** (not **mid - 1**) because **mid** itself could be the answer.

11. Complexity Analysis

Operation	Complexity
Time	$O(\log n)$
Space	$O(1)$

12. Common Patterns

Pattern	Description	Example
Insert position	Find where to insert target to maintain sorted order	LeetCode 35
Count less than target	lower_bound(arr, target) gives count of elements $<$ target	Range queries
Ceiling	arr[lower_bound(arr, target)] gives the ceiling	GFG Ceiling in sorted array

13. Common Mistakes

- Setting **hi = n - 1** instead of **hi = n** (misses the case where all elements are $<$ target).
- Using **lo $\leq$ hi** instead of **lo < hi**, causing infinite loop.
- Setting **lo = mid** instead of **lo = mid + 1** when **arr[mid] < target**.
- Forgetting that the return value can be **n** (out of bounds index).

14. Edge Cases

- **All elements < target**: Returns **n**.
- **All elements $\geq$ target**: Returns 0.
- **Empty array**: Returns 0.

- **Single element less than target:** Returns 1 (which is n).
- **Single element $\geq$ target:** Returns 0.
- **All elements equal to target:** Returns 0 (first occurrence).

15. Variations

- **Lower bound on descending array:** Reverse the comparison condition.
- **Lower bound with custom comparator:** Works with any monotonic predicate.

16. Related Algorithms

Algorithm	Connection
Upper bound	Finds first element $>$ target
First occurrence	Finds first element $=$ target (lower_bound on equality check)

3. Upper Bound

1. Overview

Upper bound finds the **first position** in a sorted array where the value is $>$ **target**. If all elements are $\leq$ target, it returns **n**. This is equivalent to C++'s `std::upper_bound`.

2. Intuition

If lower bound is "first seat where height $\geq$ target", upper bound is "first seat where height $>$ target". It's the point where values become strictly greater than the target.

3. When to Use It

- Finding the first element strictly greater than a value.
- Counting occurrences of a value: `upper_bound - lower_bound`.
- Finding the "strict ceiling" of a number.
- Range partition queries.

Common trigger phrases:

- "First element greater than ..."
- "Strictly greater than ..."
- "Count occurrences in sorted array"
- "Upper bound of ..."

4. When Not to Use It

- You need $\geq$ (non-strict) — use lower bound.
- Unsorted data.
- You need the last occurrence — use last occurrence variant.

5. Core Concepts

Concept	Explanation
Invariant	<code>arr[lo] <= target</code> on left, <code>arr[hi] > target</code> on right
Relation to lower_bound	<code>upper_bound(arr, target) == lower_bound(arr, target + 1)</code> for integers

6. Step-by-Step Algorithm

1. Initialize `lo = 0`, `hi = n`.
2. While `lo < hi`:
 - `mid = lo + (hi - lo) / 2`.
 - If `arr[mid] <= target`, set `lo = mid + 1`.
 - Else set `hi = mid`.
3. Return `lo`.

7. Dry Run

Input: `arr = [1, 3, 5, 5, 5, 7, 9]`, `target = 5`

Step	lo	hi	mid	arr[mid]	Comparison	Action
1	0	7	3	5	<code>5 <= 5</code>	<code>lo = 4</code>
2	4	7	5	7	<code>7 > 5</code>	<code>hi = 5</code>
3	4	5	4	5	<code>5 <= 5</code>	<code>lo = 5</code>
4	5	5	—	—	<code>lo == hi</code>	Stop

Result: `lo = 5`, and `arr[5] = 7` is the first element `> 5`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Returns first index where arr[idx] > target
// Returns n if no such element exists
int upperBound(const vector<int>& arr, int target) {
    int lo = 0, hi = (int)arr.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] <= target)
            lo = mid + 1;
        else
            hi = mid;
    }
    return lo;
}
```

```
// Example usage
int main() {
    vector<int> arr = {1, 3, 5, 5, 5, 7, 9};
    cout << upperBound(arr, 5) << endl;    // Output: 5 (arr[5] = 7)
    cout << upperBound(arr, 0) << endl;    // Output: 0
    cout << upperBound(arr, 10) << endl;   // Output: 7 (n)
    return 0;
}
```

9. Python Implementation

```
def upper_bound(arr, target):
    """Returns first index where arr[idx] > target.
       Returns len(arr) if no such element exists."""
    lo, hi = 0, len(arr)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] <= target:
            lo = mid + 1
        else:
            hi = mid
    return lo

# Example usage
arr = [1, 3, 5, 5, 5, 7, 9]
print(upper_bound(arr, 5))    # Output: 5
print(upper_bound(arr, 0))    # Output: 0
print(upper_bound(arr, 10))   # Output: 7
```

10. Code Explanation

- **Only difference from lower_bound:** The condition `arr[mid] <= target` (instead of `<`). When `arr[mid]` equals target, we still move right because we want strictly greater.
- Everything else (lo, hi range, loop condition, narrow logic) is identical to lower_bound.

Relation: `upper_bound(arr, target) == lower_bound(arr, target + 1)` for integer arrays.

11. Complexity Analysis

Operation	Complexity
Time	$O(\log n)$
Space	$O(1)$

12. Common Patterns

Pattern	Description	Example
Count occurrences	<code>upper_bound(target) - lower_bound(target)</code>	Counting frequency in sorted array
Range queries	Find range of indices with value == target	LeetCode 34
Strict ceiling	<code>arr[upper_bound(arr, target)]</code> if result < n	GFG

13. Common Mistakes

- Using `<` instead of `<=` in the condition (gives lower bound behavior).
- Same edge-case mistakes as `lower_bound`.

14. Edge Cases

Same as `lower_bound`, plus:

- **All elements equal to target:** Returns n.
- **Target present multiple times:** Returns first index after all occurrences.

15. Variations

- **Upper bound on descending array:** Reverse comparison.
- **Using `lower_bound` on `target+1`:** For integer targets, this is equivalent.

4. First Occurrence

1. Overview

Find the **leftmost (first) index** where a target value appears in a sorted array with duplicates. If the target is not present, return -1.

2. Intuition

Standard binary search finds *some* occurrence. When duplicates exist, we want the very first one. The trick is: when we find the target, we don't return immediately. Instead, we continue searching to the left to see if there is an earlier occurrence.

3. When to Use It

- Sorted array with duplicates and you need the first occurrence.
- Finding the start of a range of equal values.
- Exactly like `lower_bound` but for equality.

Common trigger phrases:

- "First occurrence of ..."
- "Leftmost index of ..."

- "Find first and last position"
- "Range of occurrences"

4. When Not to Use It

- No duplicates — standard binary search works fine.
- Unsorted array.
- You need any occurrence — standard binary search is simpler.

5. Core Concepts

Concept	Explanation
Post-match narrowing	After finding the target, continue searching left half to find first occurrence

6. Step-by-Step Algorithm

1. Initialize `lo = 0`, `hi = n - 1`, `ans = -1`.
2. While `lo <= hi`:
 - `mid = lo + (hi - lo) / 2`.
 - If `arr[mid] == target`: `ans = mid`, `hi = mid - 1` (search left for earlier occurrence).
 - Else if `arr[mid] < target`: `lo = mid + 1`.
 - Else: `hi = mid - 1`.
3. Return `ans`.

7. Dry Run

Input: `arr = [1, 2, 2, 2, 3, 4, 5]`, `target = 2`

Step	lo	hi	mid	arr[mid]	Action	ans
1	0	6	3	2	Found, hi = 2	3
2	0	2	1	2	Found, hi = 0	1
3	0	0	0	1	<code>arr[0] < 2</code> , <code>lo = 1</code>	1
4	1	0	—	—	<code>lo > hi</code> , stop	1

Result: `ans = 1`, which is the first occurrence.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int firstOccurrence(const vector<int>& arr, int target) {
    int lo = 0, hi = (int)arr.size() - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) {
```

```

        ans = mid;
        hi = mid - 1; // search left for earlier occurrence
    } else if (arr[mid] < target) {
        lo = mid + 1;
    } else {
        hi = mid - 1;
    }
}
return ans;
}

// Example usage
int main() {
    vector<int> arr = {1, 2, 2, 2, 3, 4, 5};
    cout << firstOccurrence(arr, 2) << endl;    // Output: 1
    cout << firstOccurrence(arr, 6) << endl;    // Output: -1
    return 0;
}

```

9. Python Implementation

```

def first_occurrence(arr, target):
    """Returns first index of target in sorted arr, or -1 if not found."""
    lo, hi = 0, len(arr) - 1
    ans = -1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target:
            ans = mid
            hi = mid - 1 # search left for earlier occurrence
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return ans

# Example usage
arr = [1, 2, 2, 2, 3, 4, 5]
print(first_occurrence(arr, 2)) # Output: 1
print(first_occurrence(arr, 6)) # Output: -1

```

10. Code Explanation

- The key difference from standard binary search: **when we find the target, we record it but keep searching left.**
- `ans` stores the latest (leftmost so far) occurrence.
- `hi = mid - 1` lets us search the left half for any earlier occurrence.

11. Complexity Analysis

Operation	Complexity
Time	$O(\log n)$
Space	$O(1)$

12. Common Patterns

Pattern	Description
First and Last Position	Combine firstOccurrence and lastOccurrence
Count occurrences	$\text{lastOccurrence} - \text{firstOccurrence} + 1$

13. Common Mistakes

- Returning immediately on finding the target (gives any occurrence, not first).
- Forgetting `ans = -1` initialization.

14. Edge Cases

- Target not present: Returns -1.
- Target appears once: Returns that index.
- Target at index 0: Works correctly.
- All elements equal to target: Returns 0.

5. Last Occurrence

1. Overview

Find the **rightmost (last) index** where a target value appears in a sorted array with duplicates. If not present, return -1.

2. Intuition

Same as first occurrence, but when we find the target, we search to the right instead of left.

3. When to Use It

- Paired with first occurrence to find the complete range of a value.
- Sorted arrays with duplicates where the rightmost position matters.

4. When Not to Use It

- No duplicates — standard binary search is simpler.
- Unsorted data.

5. Core Concepts

Concept	Explanation
Post-match rightward search	After finding target, search right half to find last occurrence

6. Step-by-Step Algorithm

1. Initialize `lo = 0`, `hi = n - 1`, `ans = -1`.
2. While `lo <= hi`:
 - `mid = lo + (hi - lo) / 2`.
 - If `arr[mid] == target`: `ans = mid`, `lo = mid + 1` (search right for later occurrence).
 - Else if `arr[mid] < target`: `lo = mid + 1`.
 - Else: `hi = mid - 1`.
3. Return `ans`.

7. Dry Run

Input: `arr = [1, 2, 2, 2, 3, 4, 5]`, `target = 2`

Step	lo	hi	mid	arr[mid]	Action	ans
1	0	6	3	2	Found, lo = 4	3
2	4	6	5	4	<code>arr[5] > 2</code> , <code>hi = 4</code>	3
3	4	4	4	3	<code>arr[4] > 2</code> , <code>hi = 3</code>	3
4	4	3	—	—	<code>lo > hi</code> , stop	3

Result: `ans = 3`, which is the last occurrence.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int lastOccurrence(const vector<int>& arr, int target) {
    int lo = 0, hi = (int)arr.size() - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) {
            ans = mid;
            lo = mid + 1; // search right for later occurrence
        } else if (arr[mid] < target) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return ans;
}

// Example usage
```

```
int main() {
    vector<int> arr = {1, 2, 2, 2, 3, 4, 5};
    cout << lastOccurrence(arr, 2) << endl; // Output: 3
    cout << lastOccurrence(arr, 6) << endl; // Output: -1
    return 0;
}
```

9. Python Implementation

```
def last_occurrence(arr, target):
    """Returns last index of target in sorted arr, or -1 if not found."""
    lo, hi = 0, len(arr) - 1
    ans = -1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target:
            ans = mid
            lo = mid + 1 # search right for later occurrence
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return ans

# Example usage
arr = [1, 2, 2, 2, 3, 4, 5]
print(last_occurrence(arr, 2)) # Output: 3
print(last_occurrence(arr, 6)) # Output: -1
```

10. Code Explanation

- **Key difference from first_occurrence:** When found, we set `lo = mid + 1` instead of `hi = mid - 1`.
- All other logic is identical to `first_occurrence`.

11. Complexity Analysis

Operation	Complexity
Time	$O(\log n)$
Space	$O(1)$

12. Common Patterns

Pattern	Description	Example
Range of a value	Combine first and last occurrence	LeetCode 34
Count occurrences	<code>last - first + 1</code>	Frequency counting

13. Common Mistakes

- Confusing with first occurrence (wrong narrowing direction after match).

14. Edge Cases

- **Target not present:** Returns -1.
 - **Target appears once:** Same index for first and last.
 - **Target at last index:** Works correctly.
-

6. Search in Rotated Sorted Array

1. Overview

Search for a target in an array that was sorted in ascending order and then rotated at some unknown pivot. E.g., [4, 5, 6, 7, 0, 1, 2]. The array has no duplicates in the classic version.

2. Intuition

A rotated sorted array consists of two sorted segments. The key insight: when you look at the midpoint, at least one half (left or right) is **completely sorted**. You can check which half is sorted, then determine if the target lies in that sorted half. If yes, search there; otherwise, search the other half.

Analogy: Imagine a sorted deck of cards that has been cut at some random point. If you pick a card in the middle, the cards from the start to that midpoint are either all in increasing order OR the cards from midpoint to end are all in increasing order. You can use that to decide which direction to search.

3. When to Use It

- Array is "rotated sorted" — a sorted array shifted at some pivot.
- You need $O(\log n)$ search.
- The problem explicitly mentions "rotated" or "shifted" sorted array.

Common trigger phrases:

- "Rotated sorted array"
- "Circularly shifted"
- "Sorted array that has been rotated"

4. When Not to Use It

- Array not rotated — use standard binary search.
- Array has duplicates (requires a modified approach — see Variation).
- You need to find the rotation point first — that's a different problem (find min in rotated sorted array).

5. Core Concepts

Concept	Explanation
Pivot	The point where the array rotates (min element index)
Sorted half	In any mid, either left half [lo..mid] or right half [mid..hi] is fully sorted
Target in sorted half check	Check if target lies within the sorted half's value range

6. Step-by-Step Algorithm

1. Initialize `lo = 0`, `hi = n - 1`.
2. While `lo <= hi`:
 - `mid = lo + (hi - lo) / 2`.
 - If `arr[mid] == target`, return `mid`.
 - If `arr[lo] <= arr[mid]` (left half is sorted):
 - If `arr[lo] <= target < arr[mid]`: `hi = mid - 1` (target in left half).
 - Else: `lo = mid + 1` (target in right half).
 - Else (right half is sorted):
 - If `arr[mid] < target <= arr[hi]`: `lo = mid + 1` (target in right half).
 - Else: `hi = mid - 1` (target in left half).
3. Return `-1`.

7. Dry Run

Input: `arr = [4, 5, 6, 7, 0, 1, 2]`, `target = 0`

Step	lo	hi	mid	arr[mid]	Sorted half	Check	Action
1	0	6	3	7	Left [4,5,6,7] sorted	0 in [4,7]? No	lo = 4
2	4	6	5	1	Right [1,2] sorted	0 in [1,2]? No	hi = 4
3	4	4	4	0	—	<code>arr[4] == 0</code>	Found at 4

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int searchRotated(const vector<int>& arr, int target) {
    int lo = 0, hi = (int)arr.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == target) return mid;

        // Left half is sorted
        if (arr[lo] <= arr[mid]) {
            if (arr[lo] <= target && target < arr[mid])
                hi = mid - 1;
            else
                lo = mid + 1;
        }
    }
}
```

```

    }
    // Right half is sorted
    else {
        if (arr[mid] < target && target <= arr[hi])
            lo = mid + 1;
        else
            hi = mid - 1;
    }
}
return -1;
}

// Example usage
int main() {
    vector<int> arr = {4, 5, 6, 7, 0, 1, 2};
    cout << searchRotated(arr, 0) << endl;    // Output: 4
    cout << searchRotated(arr, 3) << endl;    // Output: -1
    return 0;
}

```

9. Python Implementation

```

def search_rotated(arr, target):
    """Search target in rotated sorted array. Returns index or -1."""
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if arr[mid] == target:
            return mid

        # Left half is sorted
        if arr[lo] <= arr[mid]:
            if arr[lo] <= target < arr[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        # Right half is sorted
        else:
            if arr[mid] < target <= arr[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1

# Example usage
arr = [4, 5, 6, 7, 0, 1, 2]
print(search_rotated(arr, 0)) # Output: 4
print(search_rotated(arr, 3)) # Output: -1

```

10. Code Explanation

- **`arr[lo] <= arr[mid]` check:** Determines if the left half is fully sorted. If true, all elements from `lo` to `mid` are in sorted order.
- **Sorted half check:** If left half is sorted, check if target lies within `[arr[lo], arr[mid]]`. If yes, search left; otherwise search right.
- **Right half sorted:** If left is not sorted, right half must be sorted. Check if target lies in `(arr[mid], arr[hi]]`.

11. Complexity Analysis

Operation	Complexity
Time	$O(\log n)$
Space	$O(1)$

12. Common Patterns

Pattern	Description	Example
Search with rotation	Classic search	LeetCode 33
Search with duplicates	When duplicates exist, fallback to $O(n)$ in worst case	LeetCode 81
Find minimum	Find pivot/min element	LeetCode 153

13. Common Mistakes

- Forgetting the `<=` in `arr[lo] <= arr[mid]` (important when `lo == mid`).
- Incorrect bounds in the sorted-half check (using `<=` where `<` is needed, or vice versa).
- Not handling duplicates properly.

14. Edge Cases

- **Array not rotated:** Works as standard binary search.
- **Rotated at pivot 0** (full sorted): Works.
- **Two elements:** `[2, 1]` or `[1, 2]`.
- **Target at pivot point:** Works.
- **Duplicates present:** Need modification (LeetCode 81).

15. Variations

Variation	Description
With duplicates	When <code>arr[lo] == arr[mid] == arr[hi]</code> , can't determine sorted half; increment <code>lo</code> and decrement <code>hi</code>
Find minimum	Binary search for the pivot point
Find rotation count	Number of rotations = index of minimum element

7. Binary Search on Answer

1. Overview

Instead of searching for an element in an array, binary search on answer applies binary search on the **range of possible answer values** to find the optimal solution. The key requirement: the search space must be **monotonic** — if a value x is feasible, then all values greater (or lesser) are also feasible.

2. Intuition

Analogy: You're packing items into boxes, and each box has a capacity. You want to find the **minimum** box capacity needed to pack all items using at most k boxes. The answer lies in some range $[\text{min_item}, \text{sum_all}]$. If capacity C works (you can pack all items in $\leq k$ boxes), then any larger capacity also works. So the predicate "capacity C works" is monotonic. Binary search on C finds the minimum feasible capacity.

3. When to Use It

- The problem asks for "minimum possible maximum" or "maximum possible minimum".
- The answer has a clear lower and upper bound.
- The feasibility of a candidate answer can be checked in polynomial time.
- The problem asks to "minimize the maximum" or "maximize the minimum" of something.

Common trigger phrases:

- "Minimize the maximum ..."
- "Maximize the minimum ..."
- "What is the largest minimum ..."
- "What is the smallest maximum ..."
- "Split array into k subarrays minimizing the largest sum"
- "Allocate pages / books / jobs"
- "Capacity to ship packages"

4. When Not to Use It

- The predicate is not monotonic (can't use binary search).
- The answer space is small enough for linear scan.
- The feasibility check itself is too expensive (e.g., $O(n^2)$ or worse).
- The answer is not bounded or the bounds are too large.

5. Core Concepts

Concept	Explanation
Answer space	The range of possible values the answer could take
Feasibility predicate	A function $f(x)$ that returns true if x is a valid answer
Monotonicity	If $f(x)$ is true, then for all $y > x$, $f(y)$ is also true (or vice versa)
Feasibility check	The function that checks if a candidate value works

6. Step-by-Step Algorithm

1. Determine the search space: `low = minimum_possible_answer`, `high = maximum_possible_answer`.
2. While `low < high`:
 - `mid = low + (high - low) / 2`.
 - If `feasible(mid)`, set `high = mid` (search for smaller answer).
 - Else set `low = mid + 1` (need larger answer).
3. Return `low`.

(Reverse if searching for maximum feasible value.)

7. Dry Run

Problem: Split array `[7, 2, 5, 10, 8]` into `k = 2` subarrays minimizing the largest sum.

Answer range: `[max(arr), sum(arr)] = [10, 32]`

Feasibility check: Can we split into $\leq k$ subarrays with each sum $\leq mid$?

Binary search:

Step	lo	hi	mid	Feasible(mid)?	Explanation	Action
1	10	32	21	Yes	<code>[7+2+5=14, 10+8=18]</code> , both ≤ 21 , 2 subarrays	<code>hi = 21</code>
2	10	21	15	Yes	<code>[7+2+5=14, 10→stop, 8=8]</code> → 3 subarrays needed	<code>lo = 16</code>
3	16	21	18	Yes	<code>[7+2+5=14, 10+8=18]</code> , both ≤ 18 , 2 subarrays	<code>hi = 18</code>
4	16	18	17	No	<code>[7+2+5=14, 10→exceeds 17, 8=8]</code> → 3 subarrays	<code>lo = 18</code>
5	18	18	—	—	<code>lo == hi</code>	Stop

Result: 18 (minimum possible maximum subarray sum).

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Check if we can split array into <= k subarrays with max sum <= mid
bool feasible(const vector<int>& arr, int k, int mid) {
    int subarrays = 1, currentSum = 0;
    for (int x : arr) {
        if (currentSum + x > mid) {
            subarrays++;
            currentSum = x;
            if (subarrays > k) return false;
        } else {
            currentSum += x;
        }
    }
}
```

```

        return true;
    }

    // Find minimum possible maximum subarray sum when splitting into k parts
    int minimizeMaxSum(const vector<int>& arr, int k) {
        int lo = *max_element(arr.begin(), arr.end());
        int hi = accumulate(arr.begin(), arr.end(), 0);
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (feasible(arr, k, mid))
                hi = mid;
            else
                lo = mid + 1;
        }
        return lo;
    }

    // Example usage
    int main() {
        vector<int> arr = {7, 2, 5, 10, 8};
        cout << minimizeMaxSum(arr, 2) << endl; // Output: 18
        return 0;
    }

```

9. Python Implementation

```

def feasible(arr, k, mid):
    """Check if we can split array into <= k subarrays with max sum <= mid."""
    subarrays, current_sum = 1, 0
    for x in arr:
        if current_sum + x > mid:
            subarrays += 1
            current_sum = x
            if subarrays > k:
                return False
        else:
            current_sum += x
    return True

def minimize_max_sum(arr, k):
    """Find minimum possible maximum subarray sum when splitting into k parts."""
    lo, hi = max(arr), sum(arr)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if feasible(arr, k, mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

# Example usage

```

```
arr = [7, 2, 5, 10, 8]
print(minimize_max_sum(arr, 2)) # Output: 18
```

10. Code Explanation

- **lo = max(arr), hi = sum(arr)**: The answer can't be less than the largest element (that element alone needs to go somewhere) and can't exceed the sum of all elements.
- **feasible(mid)**: Greedily packs subarrays, ensuring each subarray sum $\leq$ mid. If the number of subarrays exceeds k, mid is too small.
- **Binary search direction**: Since we want the **minimum** feasible value, when feasible is true, we try smaller values (**hi = mid**). When false, we need larger values (**lo = mid + 1**).

11. Complexity Analysis

Operation	Complexity
Feasibility check	$O(n)$
Binary search iterations	$O(\log(\text{range})) = O(\log(\text{sum}(\text{arr})))$
Total	$O(n \log(\text{sum}))$

12. Common Patterns

Pattern	Description	Example
Split array k parts	Minimize largest sum	LeetCode 410
Capacity to ship packages	Minimize max weight per day	LeetCode 1011
Koko eating bananas	Minimize eating speed	LeetCode 875
Magnetic force between balls	Maximize minimum distance	LeetCode 1552
Aggressive cows	Maximize minimum distance between cows	SPOJ AGGRCOW

13. Common Mistakes

- Wrong bounds for lo and hi.
- Feasibility function is incorrect or inefficient.
- Off-by-one in narrowing: using **lo = mid** instead of **lo = mid + 1**.
- Not checking monotonicity — if the predicate isn't monotonic, binary search won't work.
- Integer overflow when computing mid for very large ranges.

14. Edge Cases

- **k = 1**: Answer is sum of all elements.
- **k = n** (each element is its own subarray): Answer is max element.
- **All elements equal**: Answer is max element * ceil(n/k).
- **Single element**: Answer is that element.
- **Large range**: Use **long long** for sums.

15. Variations

Variation	Description
Maximum possible minimum	Reverse predicate logic (check if min distance $\geq$ mid)
Minimum possible maximum	As shown above
With constraints	Feasibility may involve DP or greedy

8. Minimum Possible Maximum (MinMax)

1. Overview

A specific application of binary search on answer where the goal is to **minimize the maximum** value among partitions/groups/selections. Common in problems like "split array into k parts to minimize the largest sum."

2. Intuition

You have constraints (e.g., split into k groups), and each possible split has a "worst" group (the one with the largest sum, largest count, etc.). You want to make that worst group as good as possible — i.e., minimize the maximum.

Binary search insight: Guess a value X for the maximum. Ask: "Can I split the array such that every group's sum $\leq X$?" If yes, try a smaller X . If no, try a larger X .

3. When to Use It

- "Minimize the maximum load/sum/cost/time/distance"
- "Split into k parts to minimize the largest ..."
- "Partition array into k subarrays minimizing ..."

Common trigger phrases:

- "Minimize the largest sum"
- "Fair distribution of workload"
- "Balanced partitions"

4. When Not to Use It

- The number of partitions is not fixed.
- There's no monotonic relationship.
- DP or greedy works better for small constraints.

5-20. (Pattern follows Binary Search on Answer closely)

The key insight: this is the same as binary search on answer where the predicate checks feasibility with a given maximum, and we search for the smallest feasible maximum.

C++ template for MinMax:

```

#include <bits/stdc++.h>
using namespace std;

// Customize this for your problem
bool canPartition(const vector<int>& arr, int k, long long maxVal) {
    int count = 1;
    long long current = 0;
    for (int x : arr) {
        if (current + x > maxVal) {
            count++;
            current = x;
            if (count > k) return false;
        } else {
            current += x;
        }
    }
    return true;
}

long long minimizeMax(const vector<int>& arr, int k) {
    long long lo = *max_element(arr.begin(), arr.end());
    long long hi = accumulate(arr.begin(), arr.end(), 0LL);
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (canPartition(arr, k, mid))
            hi = mid;
        else
            lo = mid + 1;
    }
    return lo;
}

```

9. Maximum Possible Minimum (MaxMin)

1. Overview

The dual of MinMax. Here the goal is to **maximize the minimum** value (e.g., distance, value, allocation) among selections. Common in problems like "place k cows in stalls to maximize minimum distance between them."

2. Intuition

You need to place items (cows, balls, facilities) such that the minimum distance between any two is as large as possible.

Binary search insight: Guess a minimum distance D . Ask: "Can I place all k items such that every pair is at least D apart?" If yes, try a larger D . If no, try a smaller D .

Analogy: You're placing campers along a beach to give them maximum privacy. You guess a privacy distance. If you can place all campers with at least that distance between them, increase the distance. If not, decrease it.

3. When to Use It

- Place objects maximizing the minimum distance between them.
- "Maximum possible minimum" problems.
- Problems where you pick k elements from an array to maximize the minimum difference/pairwise value.

Common trigger phrases:

- "Maximize the minimum distance"
- "Largest minimum ..."
- "Place k items to maximize the minimum gap"
- "Aggressive cows"
- "Magnetic force between balls"

4. When Not to Use It

- The objective is to minimize the maximum (use MinMax instead).
- The placement order matters and is not monotonic.
- Small constraints allow direct DP.

5. Core Concepts

Concept	Explanation
Dual predicate	Check if a given minimum value is achievable
Greedy placement	Usually, place items as far apart as possible to satisfy the minimum distance
Search direction	If feasible, try larger; if not feasible, try smaller

6. Step-by-Step Algorithm

1. Sort the array (positions/distances).
2. Set $lo = 1$ (min possible distance), $hi = arr[n-1] - arr[0]$ (max possible distance).
3. While $lo < hi$:
 - $mid = lo + (hi - lo + 1) / 2$ (note: ceiling mid for max search).
 - If $feasible(mid)$, set $lo = mid$ (try larger).
 - Else set $hi = mid - 1$.
4. Return lo .

Key difference from MinMax: We use $mid = lo + (hi - lo + 1) / 2$ (ceiling) and $lo = mid$ when feasible, to avoid infinite loops.

7. Dry Run

Problem: Place 3 cows in stalls at positions $[1, 2, 8, 4, 9]$. Sort: $[1, 2, 4, 8, 9]$. Maximize minimum distance.

Feasibility: Can we place 3 cows with min distance $\geq$ mid?

Step	lo	hi	mid	Feasible?	Explanation	Action
1	1	8	4	Yes	Place at 1, 8, (none at ≥ 12) $\rightarrow$ only 2 cows	But wait, 1, 8 are ≥ 4 apart. Need 3rd. $1 \rightarrow 4 \rightarrow 8 \geq 4$? $1 \rightarrow 4 = 3 < 4$. So only 2 cows. $\rightarrow$ No
1	1	8	4	Actually No	Place 1, then 4 (diff=3<4), then 8 (diff=4 $\geq$ 4): $1 \rightarrow 8$ works, $4 \rightarrow 8$ works too. Let me redo carefully.	

Let me redo properly.

Sorted positions: [1, 2, 4, 8, 9], k = 3 cows

Feasibility function: Place first cow at 1. For each next position, if distance from last placed $\geq$ mid, place there. Count placed cows.

mid = 4: Place at 1 $\rightarrow$ next at 8 (diff=7 $\geq$ 4) $\rightarrow$ next: can we place another? From 8, need ≥ 12 . No position. Count = 2 < 3. Not feasible.

Step	lo	hi	mid	Feasible?	Action
1	1	8	4	No (only 2 cows)	hi = 3
2	1	3	2	Yes (1 $\rightarrow$ 4 $\rightarrow$ 8, 3 cows)	lo = 3
3	3	3	—	—	Stop

Result: 3 (maximum possible minimum distance).

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Check if we can place k items with min distance >= mid
bool canPlace(const vector<int>& positions, int k, int mid) {
    int count = 1;
    int lastPos = positions[0];
    for (int i = 1; i < (int)positions.size(); i++) {
        if (positions[i] - lastPos >= mid) {
            count++;
            lastPos = positions[i];
            if (count >= k) return true;
        }
    }
    return count >= k;
}
```

```
// Maximize minimum distance between k placed items
int maxMinDistance(vector<int>& positions, int k) {
    sort(positions.begin(), positions.end());
    int lo = 1, hi = positions.back() - positions.front();
    while (lo < hi) {
        int mid = lo + (hi - lo + 1) / 2; // ceiling mid
        if (canPlace(positions, k, mid))
            lo = mid; // try larger
        else
            hi = mid - 1;
    }
    return lo;
}

// Example usage
int main() {
    vector<int> stalls = {1, 2, 8, 4, 9};
    cout << maxMinDistance(stalls, 3) << endl; // Output: 3
    return 0;
}
```

9. Python Implementation

```
def can_place(positions, k, mid):
    """Check if we can place k items with min distance >= mid."""
    count = 1
    last_pos = positions[0]
    for pos in positions[1:]:
        if pos - last_pos >= mid:
            count += 1
            last_pos = pos
        if count >= k:
            return True
    return count >= k

def max_min_distance(positions, k):
    """Maximize minimum distance between k placed items."""
    positions.sort()
    lo, hi = 1, positions[-1] - positions[0]
    while lo < hi:
        mid = lo + (hi - lo + 1) // 2 # ceiling
        if can_place(positions, k, mid):
            lo = mid
        else:
            hi = mid - 1
    return lo

# Example usage
stalls = [1, 2, 8, 4, 9]
print(max_min_distance(stalls, 3)) # Output: 3
```

10. Code Explanation

- **Ceiling mid:** $\text{mid} = \text{lo} + (\text{hi} - \text{lo} + 1) / 2$ prevents infinite loops. When lo and hi differ by 1, this ensures mid moves to hi instead of staying at lo .
- **$\text{lo} = \text{mid}$ when feasible:** We want to try larger values, so we move lo up. We keep mid because mid itself might be the best answer.
- **Greedy check:** Place first item at the first position, then always place at the earliest position that satisfies the minimum distance. This greedy works because positions are sorted.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n)$
Feasibility check	$O(n)$
Binary search	$O(\log(\text{range})) = O(\log(\text{max} - \text{min}))$
Total	$O(n \log n + n \log \text{range})$

12. Common Patterns

Pattern	Description	Example
Aggressive cows	Place k cows in stalls to maximize min distance	SPOJ AGGRCOW
Magnetic force	Place k balls in baskets to maximize min force	LeetCode 1552
Minimum time to repair	Maximize minimum time allocation	Codeforces

13. Common Mistakes

- Using floor mid $(\text{lo} + \text{hi}) / 2$ instead of ceiling $(\text{lo} + \text{hi} + 1) / 2 \rightarrow$ infinite loop.
- Forgetting to sort the positions first.
- Using $\text{lo} = \text{mid} + 1$ instead of $\text{lo} = \text{mid}$ when feasible.
- Not handling $k = 1$ (answer is infinite / range max).

14. Edge Cases

- **$k = 1$:** Only one item to place; minimum distance is undefined. Answer is the full range.
- **$k = n$:** Place at every position; answer is the minimum adjacent distance.
- **All positions same:** Answer is 0.
- **Two positions only:** Answer is the difference between them.

10. Floating-Point Binary Search

1. Overview

Binary search on continuous (real-number) values. Used when the answer is a floating-point number and the predicate is monotonic. The key difference from integer binary search is that we stop based on **precision** rather than exhausting the search space.

2. Intuition

Same as integer binary search, but instead of looping until $lo \leq hi$, we loop until $hi - lo$ is smaller than some acceptable error (epsilon). Since real numbers are continuous, we never find an exact match; we find a value that is close enough.

3. When to Use It

- The answer is a real number (double/float).
- The predicate is monotonic and continuous.
- The problem specifies a tolerance/precision requirement.
- Problems involving square roots, geometric calculations, physics, etc.

Common trigger phrases:

- "Find the square root of ... up to 6 decimal places"
- "Precision of $1e-6$ "
- "Real number answer"
- "Within 10^{-6} of actual answer"
- "Floating point / decimal answer"

4. When Not to Use It

- The answer is guaranteed to be an integer — use integer binary search (faster, no precision issues).
- The problem doesn't specify precision — might accept exact fractions.
- The function is not monotonic or has discontinuities.

5. Core Concepts

Concept	Explanation
Epsilon (ϵ)	The acceptable error margin. Usually $1e-6$ or $1e-9$.
Precision vs iterations	Each iteration halves error. Need about $\log_2(\text{range}/\epsilon)$ iterations.
Fixed iterations	Often safer to run for a fixed number of iterations (e.g., 100) instead of checking $hi - lo > \epsilon$ to avoid floating-point comparison issues.

6. Step-by-Step Algorithm

1. Set $lo = \text{min_possible}$, $hi = \text{max_possible}$.
2. For 100 iterations (or while $hi - lo > 1e-6$):
 - $mid = (lo + hi) / 2$.
 - If $\text{feasible}(mid)$, set $hi = mid$ (or $lo = mid$, depending on direction).
 - Else set the opposite bound.

3. Return **lo** or **hi** (they are within ϵ of each other).

7. Dry Run

Problem: Find square root of $x = 10$, precision $1e-6$.

Iteration	lo	hi	mid	mid ²	Compare	Action
1	0	10	5.0	25	> 10	hi = 5.0
2	0	5.0	2.5	6.25	< 10	lo = 2.5
3	2.5	5.0	3.75	14.0625	> 10	hi = 3.75
4	2.5	3.75	3.125	9.765625	< 10	lo = 3.125
5	3.125	3.75	3.4375	11.816	> 10	hi = 3.4375
...	...	...	...	...	...	...
30+	3.162277	3.162278	—	≈ 10	Within ϵ	Stop

Result: ≈ 3.16227766

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Find square root of x with high precision
double sqrtBS(double x) {
    double lo = 0, hi = x;
    // Handle x < 1 (square root > x in this range conceptually...
    // Actually for x < 1, sqrt(x) > x, so hi should be 1)
    if (x < 1.0) hi = 1.0;

    for (int iter = 0; iter < 100; iter++) {
        double mid = (lo + hi) / 2;
        if (mid * mid < x)
            lo = mid;
        else
            hi = mid;
    }
    return lo;
}

// General template for floating point binary search
double floatingPointBS(double lo, double hi, function<bool(double)> feasible) {
    for (int iter = 0; iter < 100; iter++) {
        double mid = (lo + hi) / 2;
        if (feasible(mid))
            hi = mid; // or lo = mid depending on problem
        else
            lo = mid; // or hi = mid
    }
}
```

```

    }
    return lo;
}

// Example usage
int main() {
    cout << fixed << setprecision(10) << sqrtBS(10) << endl;
    // Output: 3.1622776601...
    return 0;
}

```

9. Python Implementation

```

def sqrt_bs(x, eps=1e-10):
    """Find square root of x using binary search."""
    lo, hi = 0, max(x, 1.0)
    for _ in range(100):
        mid = (lo + hi) / 2
        if mid * mid < x:
            lo = mid
        else:
            hi = mid
    return lo

# Example usage
print(f"{sqrt_bs(10):.10f}") # Output: 3.1622776601...

```

10. Code Explanation

- **Fixed iterations (100):** 100 iterations of binary search give precision of about $(\text{range}) / 2^{100}$, which is far beyond double precision. This is more reliable than checking $\text{hi} - \text{lo} > \text{eps}$.
- $\text{mid} = (\text{lo} + \text{hi}) / 2$: Simple average for floating point. No overflow risk.
- **if $(\text{mid} * \text{mid} < x)$ $\text{lo} = \text{mid}$:** When $\text{mid}^2 < x$, we need a larger value. Keep mid in the search space (unlike integer BS where we use $\text{mid} + 1$).

11. Complexity Analysis

Operation	Complexity
Per iteration	$O(1)$
Total (100 iterations)	$O(100) = O(1)$ essentially
If using ϵ -based loop	$O(\log(\text{range} / \epsilon))$

12. Common Patterns

Pattern	Description	Example
Square root	Find sqrt without library	LeetCode 69
Cube root	Binary search for cube roots	GFG
Geometric problems	Find radius, distance, angle	Codeforces
Rate calculation	Find minimum rate/time	LeetCode 1011

13. Common Mistakes

- Comparing floats with `==`. Never do `mid * mid == x`.
- Using `hi - lo > 1e-6` as loop condition — can infinite loop due to floating point precision.
- Not handling `x < 1` for square root ($\text{sqrt}(0.25) = 0.5$, which is > 0.25).
- Insufficient iterations for required precision.
- Forgetting `setprecision` or `fixed` when printing.

14. Edge Cases

- **x = 0**: $\text{sqrt}(0) = 0$.
- **x = 1**: $\text{sqrt}(1) = 1$.
- **x < 1**: $\text{sqrt}(x) > x$ for $0 < x < 1$. Need to set `hi = 1.0`.
- **Very large x**: No overflow issue with floating point.

15. Variations

Variation	Description
Fixed iterations	Run exactly 60-100 iterations regardless of range
Epsilon-based	Loop while <code>hi - lo > eps</code> — risk of infinite loop
Binary search on derivative	For finding maximum of convex functions

11. Ternary Search

1. Overview

Ternary search finds the **maximum** (or minimum) of a **unimodal** function — a function that strictly increases then strictly decreases (or vice versa). It works by dividing the search range into **three** equal parts and comparing the function values at the two division points to determine which side contains the peak.

2. Intuition

Imagine a mountain: the height increases as you climb up, reaches a peak, then decreases. You're blindfolded but can feel the slope at any point. Instead of binary search (which needs a yes/no answer), ternary search uses two probes to figure out which direction the peak lies.

How it works: Pick two points `m1` and `m2` dividing the range into three parts. Compare `f(m1)` and `f(m2)`:

- If $f(m1) < f(m2)$, the peak is between $m1$ and hi (because the function is still rising at $m1$).
- If $f(m1) > f(m2)$, the peak is between lo and $m2$.
- If equal, peak is between $m1$ and $m2$ (or both sides).

3. When to Use It

- The function is **unimodal** (single peak) on the search range.
- The function is **convex** (for minimum) or **concave** (for maximum).
- You need to find the maximum/minimum of a function without derivative.

Common trigger phrases:

- "Unimodal function"
- "Find the maximum of a function"
- "Find the minimum of a convex function"
- "Single peak"
- "Bitonic array"

4. When Not to Use It

- The function has multiple local optima (not unimodal) — use ternary search on discrete domain with caution.
- Binary search on derivative is more precise.
- The function is monotonic — use binary search.
- Discrete domain where ternary search may skip the peak.

5. Core Concepts

Concept	Explanation
Unimodal function	Strictly increasing then decreasing (or vice versa)
Probe points	Two midpoints dividing range into three equal parts
Narrowing	Discard outer third based on probe comparison

6. Step-by-Step Algorithm

For finding maximum of a unimodal function $f(x)$ in $[lo, hi]$:

1. While $hi - lo > \text{epsilon}$ (or for fixed iterations):
 - $m1 = lo + (hi - lo) / 3$
 - $m2 = hi - (hi - lo) / 3$
 - If $f(m1) < f(m2)$: peak is in $[m1, hi]$, set $lo = m1$
 - Else: peak is in $[lo, m2]$, set $hi = m2$
2. Return $(lo + hi) / 2$

7. Dry Run

Problem: Find maximum of $f(x) = -x^2 + 4x + 5$ in range $[0, 6]$. True max at $x = 2$.

Iter	lo	hi	m1	m2	f(m1)	f(m2)	Compare	New lo	New hi
1	0	6	2	4	9	5	$f(2) > f(4)$	0	4
2	0	4	1.33	2.67	8.55	8.55	$f(1.33) == f(2.67)$	1.33	2.67
3	1.33	2.67	1.78	2.22	8.95	8.95	equal	1.78	2.22
...	...	...	...	...	...	...	...	...	...

After enough iterations, $lo \approx 2.0$, $hi \approx 2.0$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Unimodal function:  $f(x) = -(x-2)^2 + 9$ 
double f(double x) {
    return -(x - 2) * (x - 2) + 9;
}

// Find maximum of unimodal function f in [lo, hi]
double ternarySearch(double lo, double hi) {
    for (int iter = 0; iter < 100; iter++) {
        double m1 = lo + (hi - lo) / 3;
        double m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2))
            lo = m1;
        else
            hi = m2;
    }
    return (lo + hi) / 2; // x of maximum
}

// For discrete ternary search on array (bitonic)
int ternarySearchArray(const vector<int>& arr) {
    int lo = 0, hi = (int)arr.size() - 1;
    while (lo < hi) {
        int m1 = lo + (hi - lo) / 3;
        int m2 = hi - (hi - lo) / 3;
        if (arr[m1] < arr[m2])
            lo = m1 + 1;
        else
            hi = m2 - 1;
    }
    return lo;
}

// Example usage
int main() {
    cout << fixed << setprecision(10);
    cout << ternarySearch(0, 6) << endl; // Output:  $\approx 2.0$ 
}
```

```
vector<int> bitonic = {1, 3, 5, 7, 9, 6, 4, 2};
cout << ternarySearchArray(bitonic) << endl; // Output: 4 (index of max=9)
return 0;
}
```

9. Python Implementation

```
def f(x):
    return -(x - 2) ** 2 + 9

def ternary_search(lo, hi):
    """Find x that maximizes f(x) in [lo, hi] (unimodal)."""
    for _ in range(100):
        m1 = lo + (hi - lo) / 3
        m2 = hi - (hi - lo) / 3
        if f(m1) < f(m2):
            lo = m1
        else:
            hi = m2
    return (lo + hi) / 2

def ternary_search_array(arr):
    """Find peak index in a bitonic array."""
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        m1 = lo + (hi - lo) // 3
        m2 = hi - (hi - lo) // 3
        if arr[m1] < arr[m2]:
            lo = m1 + 1
        else:
            hi = m2 - 1
    return lo

# Example usage
print(f"{ternary_search(0, 6):.6f}") # Output: ≈2.0
arr = [1, 3, 5, 7, 9, 6, 4, 2]
print(ternary_search_array(arr))      # Output: 4
```

10. Code Explanation

- $m1 = lo + (hi - lo) / 3$: First probe at 1/3 of the range.
- $m2 = hi - (hi - lo) / 3$: Second probe at 2/3 of the range.
- **Compare $f(m1)$ and $f(m2)$** : If $f(m1) < f(m2)$, the peak is to the right of $m1$ (since function is increasing at $m1$), so we discard $[lo, m1]$.
- **100 iterations**: Safe for double precision.
- **Discrete version**: For arrays, we use integer indices and adjust lo/hi by ± 1 to avoid infinite loops.

11. Complexity Analysis

Operation	Complexity
Continuous ternary search (100 iter)	$O(100) = O(1)$
Discrete ternary search	$O(\log_3 n) \approx O(\log n)$
Convergence rate	Each iteration reduces range by 1/3 (vs 1/2 for binary search)

12. Common Patterns

Pattern	Description	Example
Bitonic array max	Find peak in array that increases then decreases	LeetCode 852
Unimodal function	Find max/min of continuous function	Codeforces
Golden-section search	More efficient variant using golden ratio	Advanced

13. Common Mistakes

- Using ternary search on functions that are not unimodal.
- Not handling the discrete array case correctly (off-by-one).
- Using ternary search when binary search on derivative would be more efficient.
- Forgetting that each iteration reduces range by 1/3, not 1/2 — more iterations may be needed.

14. Edge Cases

- **Flat region:** If function is constant, ternary search may not converge to exact point.
- **Peak at boundary:** Works correctly; the search will narrow to the boundary.
- **Discrete with 2 elements:** Direct comparison, ternary condition may need handling.

15. Variations

Variation	Description
Golden-section search	Uses golden ratio (0.618) instead of 1/3 for optimal convergence
Discrete ternary search	For arrays with integer indices
Ternary search on derivative	Binary search on $f'(x)$ to find where $f'(x) = 0$

16. Related Algorithms

Algorithm	Connection
Binary search	For monotonic functions; ternary is for unimodal
Newton's method	Faster convergence but requires derivative
Golden-section search	More optimal ratio than 1/3

12. Binary Search with Greedy Check

1. Overview

This is the combination of **binary search on answer** with a **greedy feasibility check**. Many optimization problems reduce to: "Can we achieve X?" where the check is done greedily, and we binary search X. This is the core technique behind most MinMax/MaxMin problems.

2. Intuition

The **greedy** part is the feasibility function. For a given candidate value `mid`, we use a greedy algorithm (usually $O(n)$) to check if the target is achievable. The greedy works because the constraints are simple enough that locally optimal choices lead to globally optimal outcomes.

Analogy: Can you pack items into k boxes where each box has capacity C ? Greedy: fill each box to capacity before opening a new box. This gives the minimum number of boxes needed for capacity C . If $\text{min boxes} \leq k$, C is feasible.

3. When to Use It

- Optimization problem with a clear monotonic criterion.
- The feasibility check can be solved greedily.
- The answer space is large but the check is efficient.
- Classic "minimize maximum" and "maximize minimum" problems.

Common trigger phrases:

- (same as binary search on answer)

4. When Not to Use It

- The feasibility check requires DP or complex logic.
- The greedy doesn't work (counter-examples exist).
- The search space is small enough for linear scan.

5-20. (Covered in Binary Search on Answer sections)

The key insight: this is the same pattern as binary search on answer where the feasibility function is implemented greedily.

General template:

```
#include <bits/stdc++.h>
using namespace std;

// Greedy check: can we achieve with given mid value?
bool greedyCheck(const vector<int>& arr, int k, int mid) {
    // Typically O(n) greedy logic
    // Return true if achievable, false otherwise
```

```
}

int solve(const vector<int>& arr, int k) {
    int lo = /* minimum possible */, hi = /* maximum possible */;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (greedyCheck(arr, k, mid))
            hi = mid; // or lo = mid for max-min
        else
            lo = mid + 1;
    }
    return lo;
}
```

13. Binary Lifting Basics

1. Overview

Binary lifting (also called **binary jumping** or **doubling**) is a technique used to answer queries about **ancestors in a tree** or to process **path queries** efficiently. It precomputes "jump" pointers so that any ancestor at distance k can be found in $O(\log n)$ time.

2. Intuition

Analogy: Imagine you have a family tree and want to find the 13th ancestor of a person. Instead of walking up 13 steps one by one, you precompute: "who is my 1st, 2nd, 4th, 8th, 16th... ancestor?" Then $13 = 8 + 4 + 1$, so you jump up 8, then 4, then 1 — that's just 3 jumps instead of 13.

Key idea: Any number can be represented as a sum of powers of two (its binary representation). So instead of walking k steps one at a time, you can make at most $\log_2(k)$ jumps using precomputed 2^k -th ancestors.

3. When to Use It

- Finding the k -th ancestor of a node in a tree.
- Finding LCA (Lowest Common Ancestor) of two nodes.
- Path queries on trees (min, max, sum along a path).
- Any problem where you need to "jump" by arbitrary distances on a static structure.

Common trigger phrases:

- "K-th ancestor"
- "Lowest common ancestor"
- "Jump by distance in tree"
- "Path queries"
- "Binary lifting"

4. When Not to Use It

- The tree changes dynamically (use Heavy-Light Decomposition or Euler Tour Tree).
- k is always small (1 or 2) — linear walk is simpler.
- Memory is very tight ($O(n \log n)$ space).
- You only need LCA for a single pair — DFS + parent tracking works.

5. Core Concepts

Concept	Explanation
up[node][k]	The 2^k -th ancestor of node . up[node][0] is the parent.
LOG	$\text{Ceil}(\log_2(n))$. Typically 20 for $n \leq 10^6$.
Binary expansion	Any number k can be written as sum of powers of 2.
Preprocessing	DFS to compute $\text{up}[\text{node}][k] = \text{up}[\text{up}[\text{node}][k-1]][k-1]$

6. Step-by-Step Algorithm

Preprocessing:

1. Set $\text{LOG} = \text{ceil}(\log_2(n)) + 1$.
2. Initialize $\text{up}[\text{node}][0] = \text{parent}[\text{node}]$ (or node itself for root).
3. For $k = 1$ to $\text{LOG}-1$:
 - $\text{up}[\text{node}][k] = \text{up}[\text{up}[\text{node}][k-1]][k-1]$

K-th ancestor query:

1. For $k = \text{LOG}-1$ down to 0:
 - If $(k \gg \text{bit}) \ \& \ 1$: $\text{node} = \text{up}[\text{node}][\text{bit}]$
2. Return **node** (or -1 if root).

7. Dry Run

Tree: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ (0 is root). Find 3rd ancestor of node 4.

Precomputed table (up[node][k]):

node	up[node][0]	up[node][1]	up[node][2]
0	0	0	0
1	0	0	0
2	1	0	0
3	2	0	0
4	3	1	0

Query: $k = 3 = \text{binary } 011$

1. bit 2 (4): $3 \ \& \ 4 = 0 \rightarrow \text{skip}$
2. bit 1 (2): $3 \ \& \ 2 = 1 \rightarrow \text{node} = \text{up}[4][1] = 1$

3. bit 0 (1): $3 \& 1 = 1 \rightarrow \text{node} = \text{up}[1][0] = 0$

Result: 0 (root).

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class BinaryLifting {
    int n, LOG;
    vector<vector<int>>> up;
    vector<int> depth;

public:
    BinaryLifting(const vector<vector<int>>& adj, int root = 0) {
        n = adj.size();
        LOG = ceil(log2(n)) + 1;
        up.assign(n, vector<int>(LOG));
        depth.assign(n, 0);

        // DFS to set parent and depth
        function<void(int, int)> dfs = [&](int u, int p) {
            up[u][0] = p;
            depth[u] = (p == u ? 0 : depth[p] + 1);
            for (int k = 1; k < LOG; k++)
                up[u][k] = up[up[u][k-1]][k-1];
            for (int v : adj[u])
                if (v != p) dfs(v, u);
        };
        dfs(root, root);
    }

    // Get k-th ancestor of node u
    int kthAncestor(int u, int k) {
        if (k > depth[u]) return -1;
        for (int bit = 0; bit < LOG; bit++)
            if (k & (1 << bit))
                u = up[u][bit];
        return u;
    }

    // Get LCA of u and v
    int lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        // Bring u to same depth as v
        int diff = depth[u] - depth[v];
        for (int bit = 0; bit < LOG; bit++)
            if (diff & (1 << bit))
                u = up[u][bit];
        if (u == v) return u;
    }
};
```

```

        // Jump up together
        for (int bit = LOG - 1; bit >= 0; bit--)
            if (up[u][bit] != up[v][bit])
                u = up[u][bit], v = up[v][bit];
        return up[u][0];
    }

    int getDepth(int u) { return depth[u]; }
};

// Example usage
int main() {
    // Tree: 0-1-2-3, 0-4
    vector<vector<int>> adj = {
        {1, 4}, // 0
        {0, 2}, // 1
        {1, 3}, // 2
        {2},    // 3
        {0}     // 4
    };

    BinaryLifting bl(adj, 0);
    cout << bl.kthAncestor(3, 2) << endl; // Output: 1
    cout << bl.lca(3, 4) << endl;         // Output: 0
    return 0;
}

```

9. Python Implementation

```

class BinaryLifting:
    def __init__(self, adj, root=0):
        self.n = len(adj)
        self.LOG = (self.n).bit_length() + 1
        self.up = [[0] * self.LOG for _ in range(self.n)]
        self.depth = [0] * self.n

        # DFS to set parent and compute binary lifting table
        stack = [(root, root)]
        while stack:
            u, p = stack.pop()
            self.up[u][0] = p
            self.depth[u] = 0 if p == u else self.depth[p] + 1
            for k in range(1, self.LOG):
                self.up[u][k] = self.up[self.up[u][k-1]][k-1]
            for v in adj[u]:
                if v != p:
                    stack.append((v, u))

    def kth_ancestor(self, u, k):
        if k > self.depth[u]:
            return -1

```

```

        bit = 0
        while k:
            if k & 1:
                u = self.up[u][bit]
            k >>= 1
            bit += 1
        return u

    def lca(self, u, v):
        if self.depth[u] < self.depth[v]:
            u, v = v, u
        # Bring u to same depth
        diff = self.depth[u] - self.depth[v]
        bit = 0
        while diff:
            if diff & 1:
                u = self.up[u][bit]
            diff >>= 1
            bit += 1
        if u == v:
            return u

        for bit in range(self.LOG - 1, -1, -1):
            if self.up[u][bit] != self.up[v][bit]:
                u = self.up[u][bit]
                v = self.up[v][bit]
        return self.up[u][0]

```

10. Code Explanation

- **up[u][0] = parent[u]**: Direct parent.
- **up[u][k] = up[up[u][k-1]][k-1]**: The 2^k -th ancestor is the 2^{k-1} -th ancestor of the 2^{k-1} -th ancestor.
- **K-th ancestor**: Iterate through bits of k. For each set bit, jump by 2^{bit} .
- **LCA**: First bring both nodes to same depth, then jump up together until their parents are the same.

11. Complexity Analysis

Operation	Complexity
Preprocessing (DFS + table)	$O(n \log n)$
K-th ancestor query	$O(\log n)$
LCA query	$O(\log n)$
Space	$O(n \log n)$

12. Common Patterns

Pattern	Description	Example
K-th ancestor	Direct query	LeetCode 1483

Pattern	Description	Example
LCA	Lowest common ancestor	LeetCode 236, 235
Path min/max	Binary lifting with min/max along path	Codeforces
Distance between nodes	$\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$	Various

13. Common Mistakes

- Not handling root's ancestor queries (return -1 or root itself).
- LOG not large enough for n (should be $\text{ceil}(\log_2(n)) + 1$).
- Forgetting to set `up[root][0] = root` (self-loop for root).
- Off-by-one in `up` table indexing.

14. Edge Cases

- **Root**: k-th ancestor should return -1 or root.
- **k = 0**: Return node itself.
- **n = 1**: Only root; table has $\text{LOG} = 1$.
- **Large n**: $O(n \log n)$ memory can be heavy ($n=2e5$, $\log \approx 18$, up to 3.6M entries, fine).

15. Variations

Variation	Description
Binary lifting on functional graphs	Each node has exactly one outgoing edge
LCA with path aggregate	Store min/max/sum along 2^k jump
Parallel binary lifting	Process multiple queries simultaneously

16. Related Algorithms

Algorithm	Connection
Euler tour + RMQ	Alternative LCA method ($O(1)$ query, $O(n)$ preprocessing)
Heavy-Light Decomposition	For dynamic tree path queries
Centroid Decomposition	For distance queries on trees

14. Parallel Binary Search

1. Overview

Parallel binary search (also called **offline binary search** or **binary search on multiple queries simultaneously**) is a technique for answering many queries where each query requires a binary search. Instead of running binary search for each query independently, we process all queries together, resolving them in parallel across $\log(n)$ phases.

2. Intuition

Standard approach: For each of Q queries, run binary search independently $\rightarrow O(Q * \log(\text{range}) * \text{check_cost})$.

Parallel approach: Instead of processing queries one by one, we process all queries **simultaneously** in phases. In each phase:

1. For each query, we have a current **lo** and **hi**.
2. We compute the mid for each query.
3. We group queries by their mid value.
4. We process all queries with the same mid together.
5. We update lo/hi for each query based on results.

Analogy: Instead of baking each cake from start to finish one at a time (standard), you prepare all cakes in parallel — mix all batters in phase 1, bake in phase 2, frost in phase 3. You save setup/cleanup overhead.

3. When to Use It

- Many queries (Q) each requiring binary search on the same search space.
- The feasibility check is expensive and has common subproblems across queries.
- Offline processing is acceptable (queries are known upfront).

Common trigger phrases:

- "Multiple queries" + "binary search on answer"
- "Offline queries"
- " $Q \log N$ " too slow

4. When Not to Use It

- Q is small — standard binary search per query is fine.
- Queries are online (need immediate answers).
- The feasibility check cannot be optimized for batch processing.
- Implementation complexity is not justified by performance gain.

5. Core Concepts

Concept	Explanation
Active set	Queries still being narrowed ($lo < hi$)
Mid grouping	Queries with same mid are processed together
Batch check	Feasibility check processes all mid values for a group simultaneously
Phases	We need $O(\log(\text{highest_range}))$ phases

6. Step-by-Step Algorithm

1. For each query i : **lo[i] = min_possible**, **hi[i] = max_possible**.
2. While any query has **lo[i] < hi[i]**:
 - For each active query: **mid[i] = (lo[i] + hi[i]) / 2**.

- Group active queries by `mid[i]`.
- For each group (with value `mid`):
 - Run the feasibility check for value = `mid` across all queries.
 - For each query in group:
 - If feasible: `hi[i] = mid`
 - Else: `lo[i] = mid + 1`

3. Return `lo[i]` for each query.

7-8. (Implementation)

Parallel binary search is an advanced technique. Here's a clean implementation for the standard problem: given an array and Q queries (L, R, X), find the smallest value such that count of numbers $\leq X$ in [L, R] is $\geq K$.

```
#include <bits/stdc++.h>
using namespace std;

// Example: For each query (L, R, K), find smallest value X such that
// count of arr[L..R] <= X is at least K
// Here the "feasibility check" uses a Fenwick tree (can be any data structure)

vector<int> parallelBinarySearch(const vector<int>& arr,
                                const vector<tuple<int,int,int>>& queries) {
    int n = arr.size();
    int q = queries.size();

    vector<int> lo(q, 1), hi(q, n), ans(q);
    vector<vector<int>> byMid(n + 1); // queries grouped by mid value

    // Coordinate compress values (1..n for simplicity, assuming values 1..n)
    // In real problems, map values to ranks

    bool changed = true;
    while (changed) {
        changed = false;

        // Clear groups
        for (int i = 1; i <= n; i++) byMid[i].clear();

        // Group active queries by mid
        for (int i = 0; i < q; i++) {
            if (lo[i] < hi[i]) {
                changed = true;
                int mid = (lo[i] + hi[i]) / 2;
                byMid[mid].push_back(i);
            }
        }

        if (!changed) break;

        // Fenwick tree for batch processing
        // For each possible mid value, add appropriate elements and process
```

```

queries
    // (This part is problem-specific; below is a template structure)

    // Process mids in order (or parallelize as needed)
    FenwickTree ft(n);
    int ptr = 0;
    for (int mid = 1; mid <= n; mid++) {
        // Add elements with value == mid to Fenwick tree
        while (ptr < n && arr[ptr] <= mid) {
            ft.add(ptr + 1, 1);
            ptr++;
        }

        // Answer all queries with this mid
        for (int idx : byMid[mid]) {
            auto [L, R, K] = queries[idx];
            if (ft.sum(R) - ft.sum(L - 1) >= K) {
                hi[idx] = mid;
            } else {
                lo[idx] = mid + 1;
            }
        }
    }

    for (int i = 0; i < q; i++) ans[i] = lo[i];
    return ans;
}

```

9. Python Implementation

```

def parallel_binary_search(arr, queries):
    """Parallel binary search. queries = [(L, R, K), ...].
    Find smallest X with count of arr[L:R] <= X >= K."""
    n, q = len(arr), len(queries)
    lo, hi = [1] * q, [n] * q
    ans = [0] * q

    while True:
        changed = False
        by_mid = [[] for _ in range(n + 1)]

        for i in range(q):
            if lo[i] < hi[i]:
                changed = True
                mid = (lo[i] + hi[i]) // 2
                by_mid[mid].append(i)

        if not changed:
            break

```

```

# Process mids (Fenwick tree simulation)
ft = [0] * (n + 2)

def fenwick_add(i, delta):
    i += 1
    while i <= n + 1:
        ft[i] += delta
        i += i & -i

def fenwick_sum(i):
    i += 1
    s = 0
    while i > 0:
        s += ft[i]
        i -= i & -i
    return s

ptr = 0
for mid in range(1, n + 1):
    while ptr < n and arr[ptr] <= mid:
        fenwick_add(ptr, 1)
        ptr += 1

    for idx in by_mid[mid]:
        L, R, K = queries[idx]
        count = fenwick_sum(R) - fenwick_sum(L - 1)
        if count >= K:
            hi[idx] = mid
        else:
            lo[idx] = mid + 1

return lo

```

10. Code Explanation

- **Outer loop:** Runs $O(\log \text{range})$ times (typically 17-20 iterations).
- **Grouping:** Each iteration groups queries by their mid value.
- **Batch processing:** For each mid, we process all queries with that mid in one batch, reusing the data structure efficiently.
- **Fenwick tree:** A common data structure for parallel binary search because it can be built incrementally.

11. Complexity Analysis

Operation	Complexity
Each phase	$O((n + Q) * \log n)$ with Fenwick tree
Number of phases	$O(\log \text{range})$
Total	$O((n + Q) * \log n * \log \text{range})$
Without parallelization	$O(Q * \log n * \log \text{range})$

Parallel binary search shines when the feasibility check for different mids can share computation (e.g., adding elements incrementally to a Fenwick tree).

12. Common Patterns

Pattern	Description	Example
Meteors (IOI)	Circular array, queries assign value, find when each reaches threshold	IOI 2012
K-th smallest in range	Find K-th smallest element in subarray with many queries	Codeforces
Online judge problems	Problems with multiple queries needing binary search	Various CP

13. Common Mistakes

- Not using **while** loop correctly for converging lo/hi.
- Grouping overhead dominating the complexity.
- Incorrect mid calculation (off-by-one).
- Not resetting data structure between phases properly.

14. Edge Cases

- **Single query**: Degenerates to standard binary search.
- **All queries have same answer**: Still goes through all phases.
- **No active queries early**: Break early.
- **Large Q and n**: Must use efficient data structures.

15. Variations

Variation	Description
Divide and Conquer on queries	Recursively split queries by answer value
Parallel binary search + DSU	For graph connectivity problems
Online → Offline	Convert online queries to offline for PBP

16. Related Algorithms

Algorithm	Connection
Standard binary search	Per-query binary search
Divide and Conquer	PBP is a form of divide and conquer on queries
Mo's algorithm	Another offline query processing technique

15. Parametric Search (Advanced)

1. Overview

Parametric search is a powerful technique where the **decision problem** (feasibility check) itself involves an optimization problem, typically solved by another algorithm. The parameters of the decision algorithm become the target of binary search.

2. Intuition

Standard binary search on answer: The feasibility check is simple (greedy $O(n)$).

Parametric search: The feasibility check itself is complex — it might be a graph algorithm, DP, or flow network. The parameter we're binary-searching affects the behavior of this algorithm.

Analogy: You're designing a transportation network. You want to know the minimum budget needed to connect all cities. Your feasibility check: "Given budget B , can I connect all cities?" This involves running a minimum spanning tree algorithm (MST) with the constraint that only roads costing $\leq B$ can be built. The budget B is the parameter being binary-searched, and the feasibility check runs MST.

3. When to Use It

- The decision version of the problem is easier than the optimization version.
- The feasibility check is itself a non-trivial algorithm (graph, DP, flow).
- The answer is monotonic with respect to the parameter.
- Classic problems: find the minimum/maximum value of a parameter that makes a complex condition feasible.

Common trigger phrases:

- "Find the minimum X such that..."
- "Decision version of an optimization problem"
- "Parameterized feasibility with complex check"

4. When Not to Use It

- Simple greedy check works — use standard binary search on answer.
- The decision problem is as hard as the optimization problem.
- The monotonicity is not obvious or provable.
- The feasibility check is too expensive to run even $\log(\text{range})$ times.

5. Core Concepts

Concept	Explanation
Decision problem	A simpler version: "Is value X feasible?"
Optimization → Decision	Convert "find maximum X " to "can we achieve X ?"
Monotonicity	If X is feasible, all $Y > X$ (or $Y < X$) are also feasible
Parameterized algorithm	The algorithm inside the check uses the parameter as input

6-20. (This is an extension of Binary Search on Answer)

The key difference is **what the feasibility function does** — in parametric search, it's more complex.

General template for parametric search:

```
#include <bits/stdc++.h>
using namespace std;

// Complex feasibility check - could involve graph, DP, flow, etc.
bool feasible(int parameter) {
    // Run a complex algorithm that depends on parameter
    // Return true/false

    // Example: MST with constraint that only edges with weight <= parameter can
    // be used
    // Run Kruskal's, check if graph becomes connected
}

int parametricSearch(int lo, int hi) {
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (feasible(mid))
            hi = mid; // or lo = mid for maximize
        else
            lo = mid + 1;
    }
    return lo;
}
```

Example problem: Find the minimum fuel capacity such that a car can travel across all cities (edges with fuel cost > capacity cannot be used). Feasibility check: run DFS/BFS/DSU with only edges having weight $\leq$ fuel capacity to see if graph is connected.

Final Cheat Sheet

BINARY SEARCH - COMPLETE CHEAT SHEET

BASIC BINARY SEARCH

Use: Sorted array, find exact element

```
Code: while (lo <= hi) { mid = lo + (hi-lo)/2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) lo = mid+1; else hi = mid-1; }
```

Comp: $O(\log n)$ time, $O(1)$ space

LOWER BOUND / UPPER BOUND

Lower: first idx where $\text{arr}[\text{idx}] \geq \text{target}$ ($\text{hi} = \text{n}$, $\text{lo} < \text{hi}$ loop)

Upper: first idx where $\text{arr}[\text{idx}] > \text{target}$ ($\text{arr}[\text{mid}] \leq \text{target}$ condition)

Use: Insertion point, counting, range queries

FIRST / LAST OCCURRENCE

First: $\text{ans} = -1$; if found: $\text{ans} = \text{mid}$, $\text{hi} = \text{mid} - 1$ (search left)

Last: if found: $\text{ans} = \text{mid}$, $\text{lo} = \text{mid} + 1$ (search right)

Use: Sorted array with duplicates

ROTATED SORTED ARRAY

Check: if $\text{arr}[\text{lo}] \leq \text{arr}[\text{mid}]$ (left sorted) or $\text{arr}[\text{mid}] < \text{arr}[\text{hi}]$

Then: check if target in sorted half

Use: Search in rotated array (LeetCode 33)

BINARY SEARCH ON ANSWER (MinMax / MaxMin)

```

Template: while (lo < hi) {
    mid = lo + (hi - lo) / 2;          // For MinMax
    // mid = lo + (hi - lo + 1) / 2; // For MaxMin (ceiling)
    if (feasible(mid)) hi = mid;      // For MinMax
    else lo = mid + 1;
}

```

Feasibility: greedy/ $O(n)$ check if candidate works

FLOATING POINT BS

```

for (int iter = 0; iter < 100; iter++) {
    mid = (lo + hi) / 2; if (f(mid) < target) lo = mid; else hi = mid; }

```

Use: sqrt, real number answers

TERNARY SEARCH

$\text{m1} = \text{lo} + (\text{hi} - \text{lo}) / 3$; $\text{m2} = \text{hi} - (\text{hi} - \text{lo}) / 3$;

if ($\text{f}(\text{m1}) < \text{f}(\text{m2})$) $\text{lo} = \text{m1}$; else $\text{hi} = \text{m2}$;

Use: Unimodal function (peak/valley)

BINARY LIFTING

$\text{up}[\text{u}][\text{k}] = \text{up}[\text{up}[\text{u}][\text{k}-1]][\text{k}-1]$;

K-th ancestor: iterate bits of k, jump

LCA: bring to same depth, then jump together

Comp: $O(n \log n)$ preprocess, $O(\log n)$ query

PARALLEL BINARY SEARCH

Phase approach: group queries by mid, process in batch

Use: Many queries, expensive check, offline

PARAMETRIC SEARCH

Same as BS on answer, but feasibility runs complex algorithm
Use: Decision problem easier than optimization

COMMON TRAPS

- Off-by-one: $lo \leq hi$ vs $lo < hi$
- $hi = n$ vs $hi = n-1$ for lower_bound
- Overflow: $mid = lo + (hi-lo)/2$ (not $(lo+hi)/2$)
- Infinite loop: using $lo = mid$ without ceiling in MaxMin
- Not sorting before binary search
- Wrong mid formula for MaxMin (use $+1$)

Pro tip: Most binary search on answer problems follow one of two templates:

1. **Minimize X** → `if (feasible(mid)) hi = mid; else lo = mid + 1;` with floor mid.
2. **Maximize X** → `if (feasible(mid)) lo = mid; else hi = mid - 1;` with ceiling mid
`(lo+hi+1)/2.`

Memorize these two patterns and you can solve 90% of binary search on answer problems.